Trabajo Práctico Integrador – Seguridad en Sistemas Operativos

Control de acceso, auditoría y delegación de privilegios en Linux (Debian GNU/Linux 13)

Dato	Detalle
Alumno	Ramirez, Cristian – crishtrr751@gmail.com
Materia	Arquitectura y Sistemas Operativos
Profesor/a	Nicolas Carcaño
Fecha de entrega	9/11/2026

Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía
8. Anexos

1. Introducción

La seguridad en los sistemas operativos es un aspecto fundamental para proteger la información y los recursos de un sistema frente a accesos no autorizados. Para un técnico en programación, comprender cómo se controlan los permisos, cómo se delegan privilegios y cómo se registran los eventos resulta esencial, ya que cualquier aplicación o servidor que desarrolle o administre se ejecuta sobre un sistema operativo que debe estar correctamente configurado.

Se eligió este tema por su relevancia práctica y porque permite observar, con pocos comandos, mecanismos que utilizan a diario todos los sistemas multiusuario.

Los objetivos del trabajo son: (a) comprender los principales mecanismos de protección de un sistema operativo; (b) aplicar restricciones de acceso a un archivo mediante permisos, usuarios y grupos en Linux; (c) registrar los accesos mediante auditoría con `auditd`; y (d) verificar el uso controlado de privilegios administrativos con `sudo`.

2. Marco Teórico

Los sistemas operativos modernos incorporan mecanismos para controlar el acceso a archivos, usuarios, procesos y dispositivos. La **protección** se ocupa de controlar el acceso de procesos y usuarios a los recursos definidos por el sistema, mientras que la **seguridad** abarca además la defensa frente a amenazas externas e internas (Silberschatz et al., 2018).

Entre los conceptos clave se incluyen:

2.1 Usuarios y grupos

Cada usuario posee una identidad (UID) y pertenece a uno o más grupos (GID). La segmentación por identidad permite otorgar o negar acceso a los recursos según quién los solicita. En Linux, la información se almacena en `/etc/passwd`, `/etc/group` y `/etc/shadow`.

2.2 Permisos de archivos

Cada archivo tiene un propietario, un grupo y tres conjuntos de permisos: para el **usuario** (propietario), el **grupo** y **otros**. Cada conjunto combina lectura (r = 4), escritura (w = 2) y ejecución (x = 1), y suele expresarse en notación octal (Shotts, 2019).

Notación octal	Simbólica	Significado
600	rw-------	Solo el propietario lee y escribe
640	rw-r-----	Propietario lee y escribe; el grupo solo lee; otros no acceden
644	rw-r--r--	Todos leen; solo el propietario escribe
755	rwxr-xr-x	Todos leen y ejecutan; solo el propietario escribe

2.3 Control de acceso basado en roles (RBAC)

En RBAC los permisos no se asignan a personas individuales sino a **roles**, y los usuarios obtienen sus permisos al ser asignados a un rol. En Linux, una aproximación habitual se logra con grupos (por ejemplo, un grupo `datos` con acceso de lectura a cierta información) y con la configuración de `sudoers`. Existen además sistemas de control de acceso obligatorio como SELinux y AppArmor (este último habilitado por defecto en Debian).

2.4 Manejo de privilegios: sudo y setuid

`sudo` permite que usuarios autorizados ejecuten comandos con privilegios de administrador (root) de forma controlada, quedando el uso registrado. La configuración reside en `/etc/sudoers`. El bit **setuid** hace que un

programa se ejecute con los privilegios de su propietario y no con los de quien lo invoca; por ejemplo, `passwd` necesita modificar `/etc/shadow`, archivo que solo root puede escribir.

2.5 Auditoría de eventos

La auditoría registra eventos relevantes para la seguridad, lo que permite detectar y analizar incidentes. En Linux, el sistema de auditoría (`auditd`) se configura mediante reglas con `auditctl` y se consulta con `ausearch`. Una regla de vigilancia sobre un archivo (*watch*) registra los accesos de lectura, escritura, ejecución y cambios de atributos (Linux Audit Project, s. f.).

3. Caso Práctico

3.1 Descripción del problema

Se simuló la situación de una pequeña organización que posee un archivo confidencial al que no todos los usuarios deben acceder. Se definieron los siguientes requisitos:

Usuario	**Rol**	**Acceso esperado a `/seguro/datos.txt`**
`usuario1`	Usuario común, sin grupo especial	Ninguno
`analista`	Miembro del grupo `datos`	Solo lectura
`cris`	Administrador, pertenece al grupo `sudo`	Acceso total mediante `sudo`

Además, todo intento de acceso al archivo debía quedar registrado en el log de auditoría.

El entorno fue una máquina virtual de Oracle VirtualBox 7.2.22 con Debian GNU/Linux 13 (trixie), 2048 MB de RAM y 1 CPU, ejecutada sobre un equipo con Windows 10. Como el portapapeles compartido de la VM no estaba disponible, se accedió a ella por SSH desde PowerShell mediante reenvío de puertos de VirtualBox (puerto 2222 del equipo anfitrión hacia el puerto 22 de la VM). Así se pudo copiar y pegar comandos y obtener todas las capturas desde una misma ventana.

3.2 Conexión a la VM e instalación de auditd

```
ssh -p 2222 cris@127.0.0.1
lsb_release -a sudo apt install -y
auditd sudo systemctl enable --now
auditd systemctl status auditd --
no-pager
```

El comando `lsb_release -a` confirmó el sistema: Debian GNU/Linux 13 (trixie). El aviso «No LSB modules are available» es normal en Debian y no afecta al trabajo. Tras `systemctl enable --now`, el servicio `auditd` quedó activo (`active (running)`).

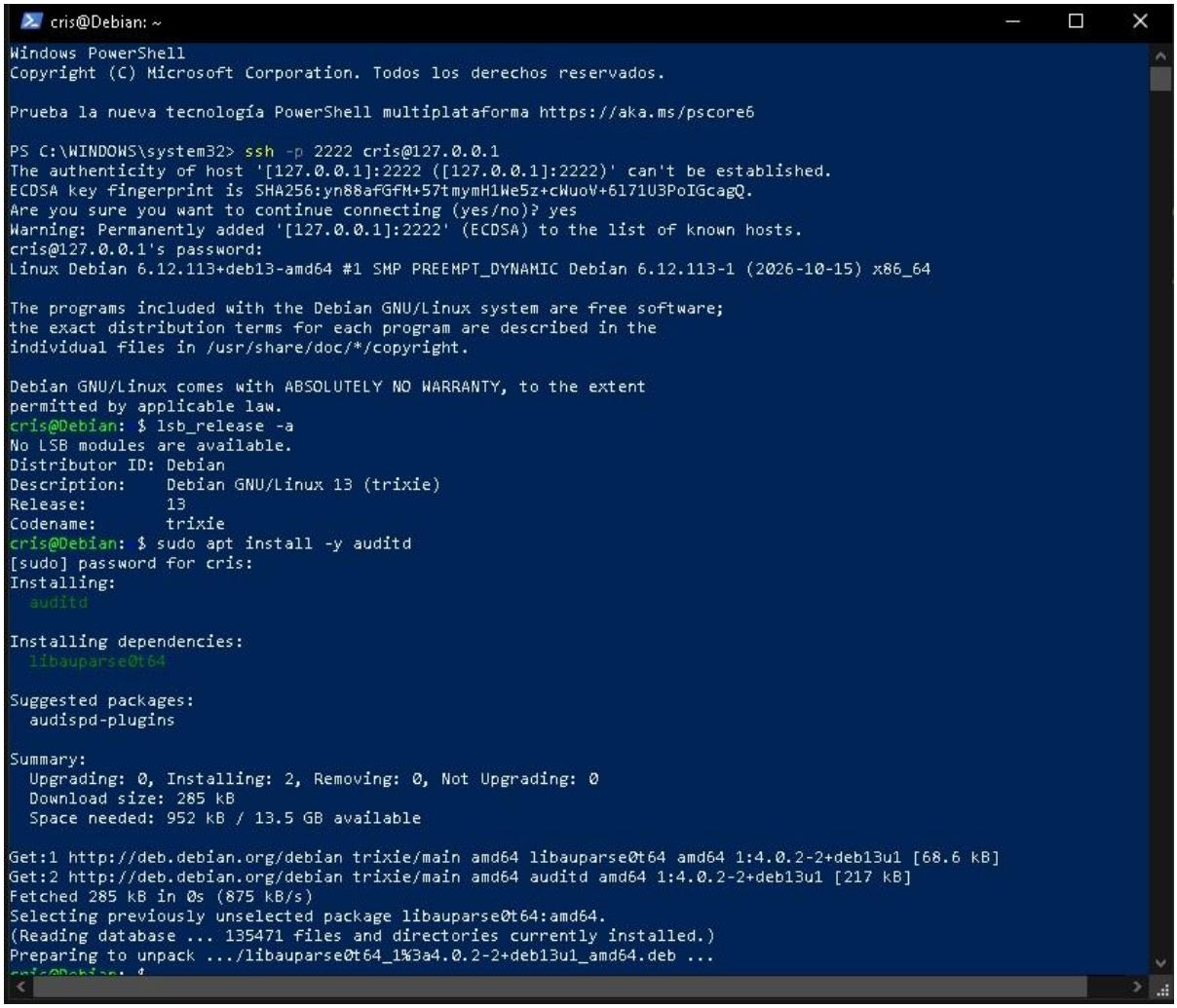


Figura 1. Conexión por SSH, versión del sistema (`lsb_release -a`) e inicio de la instalación de `auditd`.

3.3 Creación de usuarios y grupo

```
sudo useradd -m -s /bin/bash usuario1
sudo passwd usuario1
sudo useradd -m -s /bin/bash analista
sudo passwd analista sudo groupadd
datos
sudo usermod -aG datos analista id
analista
```

Se asignaron contraseñas de prueba a ambos usuarios. El comando `id analista` confirma que `analista` pertenece a su grupo primario y al grupo `datos`.



Figura 2. Pertenencia de `analista` al grupo `datos` (`id analista`).

3.4 Creación del archivo protegido y asignación de permisos

```
sudo mkdir -p /seguro
echo "informacion confidencial" | sudo tee /seguro/datos.txt
sudo chown root:datos /seguro/datos.txt sudo chmod 640
/seguro/datos.txt ls -l /seguro
```

El modo 640 otorga lectura y escritura al propietario (`root`), solo lectura al grupo `datos` y ningún permiso al resto de los usuarios. La salida muestra `-rw-r----- 1 root datos 25 Oct 22 11:50 datos.txt`.

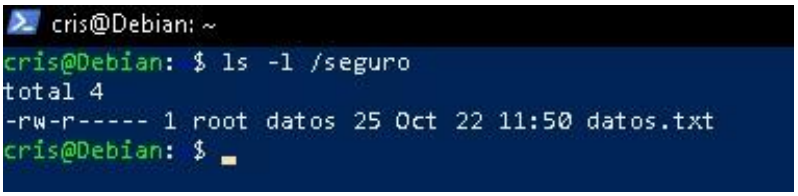


Figura 3. Propietario, grupo y permisos de `/seguro/datos.txt` (`ls -l /seguro`).

3.5 Activación de la auditoría sobre el archivo

```
sudo auditctl -w /seguro/datos.txt -p rwxa -k acceso_datos sudo
auditctl -l
```

La opción `-w` indica el archivo a vigilar, `-p rwxa` los tipos de acceso a registrar (lectura, escritura, ejecución y cambio de atributos) y `-k` la clave que identifica los eventos para su búsqueda posterior. La regla quedó cargada, como lo confirman los eventos `CONFIG_CHANGE` del registro (Figura 5).

3.6 Pruebas de acceso

```
sudo -u usuario1 cat /seguro/datos.txt sudo -u
analista cat /seguro/datos.txt
sudo -u analista sh -c 'echo prueba >>
/seguro/datos.txt'
```

El resultado fue el esperado: `usuario1` recibió «Permission denied» al intentar leer el archivo; `analista` pudo leerlo y vio «informacion confidencial»; y `analista` recibió «Permission denied» al intentar escribir en él.

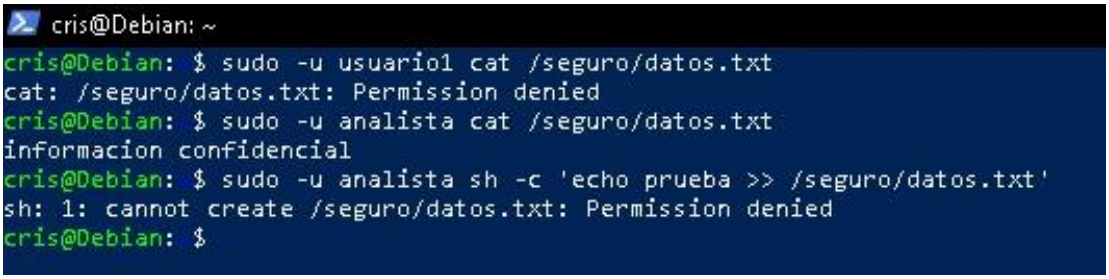


Figura 4. Pruebas de acceso al archivo con `usuario1` y `analista`.

3.7 Consulta del registro de auditoría

```
sudo ausearch -k acceso_datos -i | tail -n 30
```

En la salida, el campo `auid` identifica al usuario que inició sesión originalmente (`cris` en todos los eventos), `uid` al usuario con el que se ejecutó el comando, y `success` indica si el acceso fue permitido (`yes`) o denegado (`no`). Los tres accesos de la sección anterior quedaron registrados con la clave `acceso_datos`:

Hora	uid	Comando	success
12:07:57	usuario1	cat /seguro/datos.txt	no (EACCES, permiso denegado)
12:09:33	analista	cat /seguro/datos.txt	yes
12:09:46	analista	sh -c 'echo prueba >> /seguro/datos.txt'	no (EACCES, permiso denegado)

Además aparecen dos eventos `CONFIG_CHANGE` de `auditctl` (11:53:08 con `res=yes` y 11:53:25 con `res=no`): el segundo corresponde a un intento repetido de cargar la regla, que ya existía. La captura del registro completo (Figura 5) se muestra en la página siguiente, en horizontal para que pueda leerse.

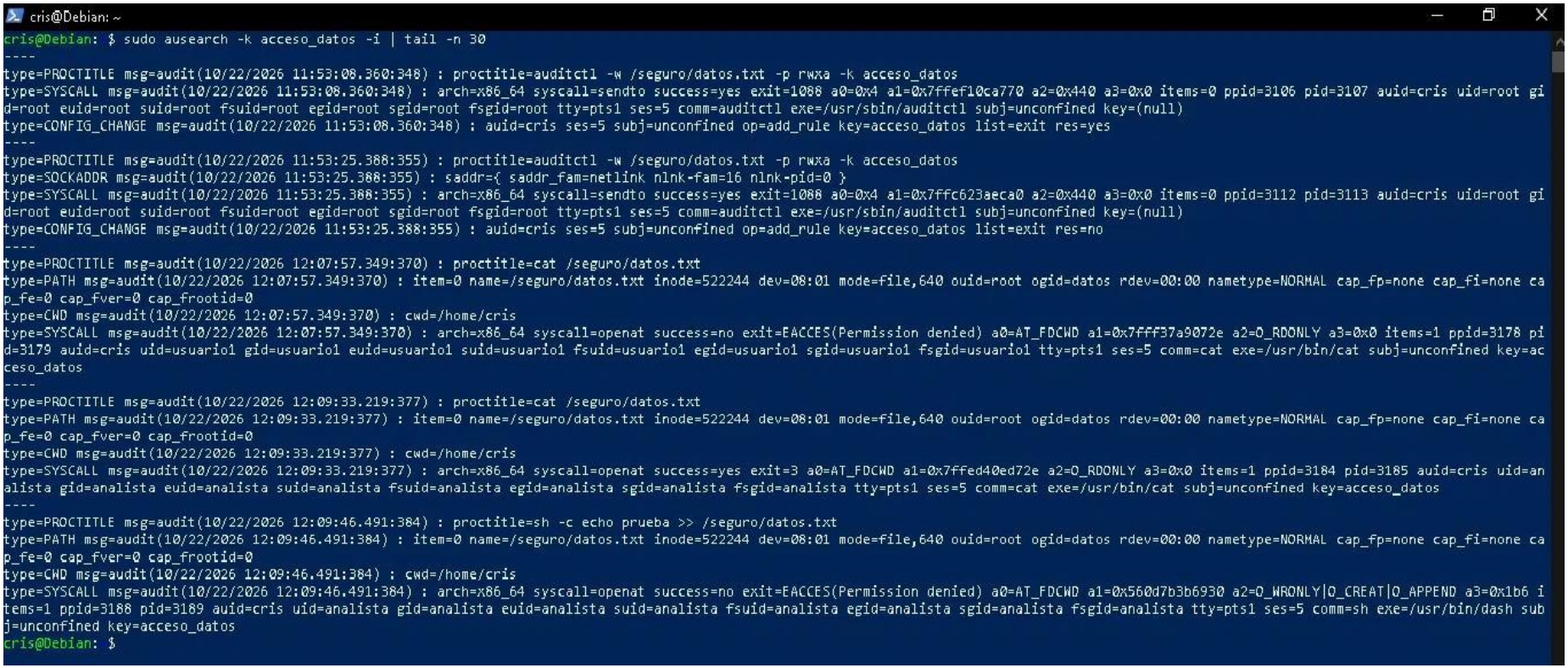


Figura 5. Registro de auditoría con la clave `acceso_datos` (`ausearch -k acceso_datos -i`).

3.8 Uso de sudo y delegación de privilegios

Primero se probó con un usuario sin privilegios:

```
su - usuario1
sudo ls /root
exit
```

Al ingresar su contraseña, `usuario1` obtuvo el mensaje «usuario1 is not in the sudoers file.».

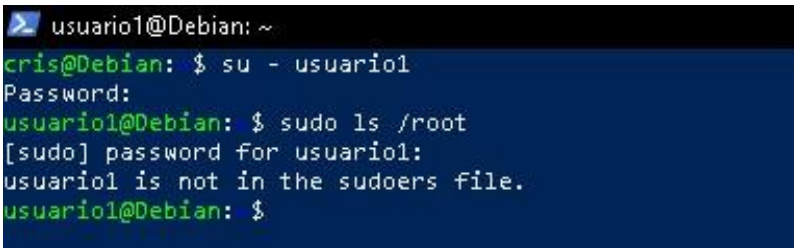


Figura 6. Intento de `usuario1` de usar `sudo`, rechazado.

Luego, con el usuario administrador `cris`:

```
sudo ls /root
groups
sudo -l -U analista sudo journalctl -t sudo --
no-pager | tail -n 5
```

`cris` pudo listar `/root` (apareció `vboxpostinstall.sh`), `groups` muestra que pertenece al grupo `sudo`, y `sudo -l -U analista` indica que `analista` no tiene permitido ejecutar `sudo` en el equipo. El registro de `journalctl` muestra las sesiones de `sudo` abiertas por `cris`.

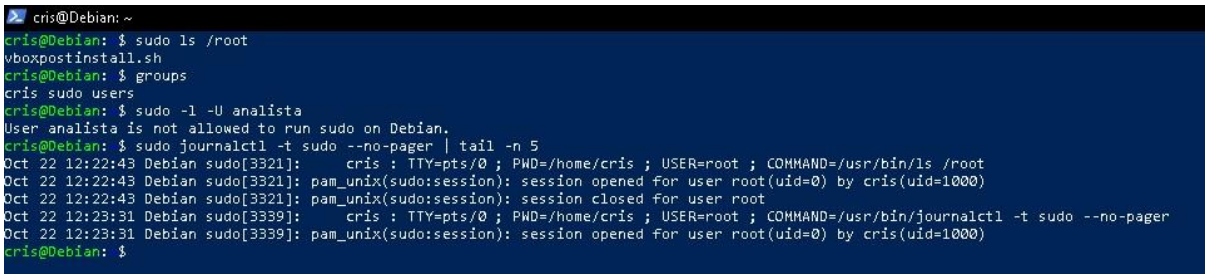


Figura 7. Uso de `sudo` por `cris`, grupos del usuario y registro de `sudo`.

3.9 Ejemplo de setuid

Para ilustrar el mecanismo se inspeccionaron los permisos del comando `passwd`:

```
ls -l /usr/bin/passwd
```

La salida es `-rwsr-xr-x 1 root root 118168 Apr 19 2025 /usr/bin/passwd`. La letra `s` en la posición de ejecución del propietario indica que el bit setuid está activo: al ejecutar `passwd`, el programa corre con los privilegios de `root`, lo que le permite modificar `/etc/shadow`.

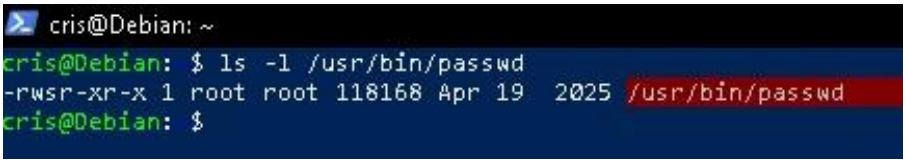


Figura 8. Bit setuid en `/usr/bin/passwd`.

3.10 Persistencia de la regla de auditoría

Las reglas cargadas con `auditctl` se pierden al reiniciar. Para hacerla permanente se guardó la regla en `/etc/audit/rules.d/` y se recargó la configuración:

```
echo '-w /seguro/datos.txt -p rwxa -k acceso_datos' | sudo tee
/etc/audit/rules.d/acceso.rule
s sudo augenrules --load sudo
auditctl -l
sudo cat /etc/audit/rules.d/acceso.rules
```

Leer el archivo de reglas requiere privilegios de administrador: sin `sudo`, el sistema devuelve «Permission denied». Con `sudo` se verifica que el archivo contiene la regla.

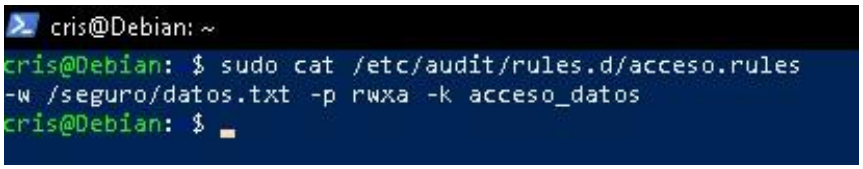


Figura 9. Regla guardada en `/etc/audit/rules.d/acceso.rules`.

4. Metodología Utilizada

- **Investigación previa:** se consultaron las páginas del manual de `chmod`, `sudoers`, `auditctl` y `ausearch`, la documentación de Debian y la bibliografía de la materia (ver sección 7).
- **Herramientas y recursos:** Oracle VirtualBox 7.2.22 sobre Windows 10; Debian GNU/Linux 13 (trixie) con 2048 MB de RAM y 1 CPU; PowerShell con el cliente SSH de Windows (reenvío de puertos 2222 → 22); `auditd` y utilidades estándar de Linux (`chmod`, `chown`, `useradd`, `usermod`, `sudo`); la herramienta de recortes de Windows para las capturas.
- **Trabajo realizado:** el trabajo fue desarrollado de forma individual por Cristian Ramirez, quien se ocupó de la configuración del entorno, las pruebas, las capturas y la redacción del informe.

El desarrollo siguió estas etapas:

1. Instalación de la máquina virtual con Debian 13 en VirtualBox.
2. Acceso por SSH desde PowerShell, para poder copiar y pegar comandos.

3. Instalación y habilitación de `auditd`.
4. Creación de los usuarios `usuario1` y `analista`, y del grupo `datos`.
5. Creación del archivo `/seguro/datos.txt` y asignación de permisos (640, propietario `root`, grupo `datos`).
6. Activación de la regla de auditoría sobre el archivo.
7. Pruebas controladas con cada usuario, consulta del registro y prueba de `sudo`.
8. Verificación del bit setuid y persistencia de la regla de auditoría.
9. Documentación de comandos, resultados, capturas y errores.

5. Resultados Obtenidos 5.1

Resumen de pruebas

Prueba	Resultado esperado	Resultado observado
`usuario1` lee `/seguro/datos.txt`	Permiso denegado	«Permission denied» (Figura 4)
`analista` lee el archivo	Muestra el contenido	Mostró «informacion confidencial» (Figura 4)
`analista` escribe en el archivo	Permiso denegado	«Permission denied» (Figura 4)
Intento de `usuario1` en el registro	Evento con `success=no` y clave `acceso_datos`	Evento de las 12:07:57 con `success=no` y `key=acceso_datos` (Figura 5)
`usuario1` ejecuta `sudo ls /root`	Rechazado: no figura en sudoers	«usuario1 is not in the sudoers file.» (Figura 6)
`cris` ejecuta `sudo ls /root`	Permitido	Listó el contenido de `/root` (Figura 7)
`sudo -l -U analista`	`analista` sin permisos de `sudo`	«User analista is not allowed to run sudo on Debian.» (Figura 7)
Permisos de `/usr/bin/passwd`	Bit setuid activo	`-rwsr-xr-x`, propietario `root` (Figura 8)
Regla de auditoría persistente	Archivo en `/etc/audit/rules.d/` con la regla	El archivo `acceso.rules` contiene la regla (Figura 9)

5.2 Aspectos que funcionaron correctamente

- Se validó el bloqueo de acceso al archivo protegido: `usuario1` recibió «Permission denied» al intentar leerlo.
- Se comprobó el principio de mínimo privilegio: `analista` pudo leer el archivo pero no modificarlo.
- Se registraron los intentos de acceso en el log de auditoría con la clave `acceso_datos`, tanto los permitidos como los denegados, y se consultaron con `ausearch`.
- Se verificó la delegación de privilegios con `sudo`: solo el usuario del grupo `sudo` pudo ejecutar comandos administrativos.
- La regla de auditoría quedó guardada en `/etc/audit/rules.d/` para que se cargue en cada arranque.

5.3 Dificultades y errores corregidos

Dificultad	Causa	Solución
No se podía pegar comandos en la terminal de la VM	El portapapeles compartido no estaba disponible. Al intentar instalar `virtualbox-guest-x11`, `apt` informó que no se podía localizar el paquete	Se instaló `openssh-server` y se accedió por SSH desde PowerShell con reenvío de puertos (2222 → 22), donde sí se puede pegar
Aviso de error en `sudo apt update`	El instalador dejó el CD de instalación de Debian como fuente de paquetes (repositorio `cdrom`)	Los repositorios de internet se actualizaron igual y `apt` siguió funcionando; el aviso no afectó al trabajo
Error «se interrumpió la ejecución de dpkg»	Una instalación anterior quedó incompleta	Se ejecutó `sudo dpkg --configure -a`, que terminó la configuración pendiente
Comandos que no se ejecutaban o salían mezclados	Al pegar varias líneas, `sudo` pedía la contraseña y absorbía los comandos siguientes. Por eso `analista` no figuraba en el grupo `datos` y el archivo `datos.txt` no se había creado (`ls -l /seguro` mostraba `total 0`)	Se repitieron los comandos de a uno, esperando que volviera el prompt
«Rule exists» al cargar la regla de auditoría	La regla ya estaba cargada de un pegado anterior	Se comprobó con `sudo auditctl -l` que la regla estaba activa

«Authentication failure» en `su - usuario1`	La contraseña de `usuario1` no había quedado asignada como se esperaba	Se volvió a ejecutar `sudo passwd usuario1` y se repitió la prueba

6. Conclusiones

El control de acceso y la auditoría son herramientas claves para prevenir y detectar incidentes. La práctica permitió observar cómo configuraciones pequeñas, como el modo de un archivo o la pertenencia a un grupo, refuerzan de manera concreta la seguridad de un sistema operativo, y cómo la auditoría deja evidencia de lo ocurrido incluso cuando el acceso fue denegado.

También se comprendió el principio de mínimo privilegio, al dar a `analista` solo lectura sobre el archivo, y la diferencia entre `auid` (quien inició sesión) y `uid` (el usuario con el que se ejecutó el comando) en el registro de auditoría. Además se comprobó que `sudo` solo está disponible para los usuarios autorizados y que el bit setuid permite que un programa como `passwd` modifique archivos protegidos.

Las dificultades principales fueron operativas: el portapapeles de la VM y los comandos que se mezclaban al pegar con `sudo`. Se resolvieron accediendo por SSH desde PowerShell y ejecutando los comandos de a uno.

Como mejoras o extensiones futuras se proponen utilizar listas de control de acceso (ACL) para permisos más finos, restringir `sudo` a comandos específicos mediante `sudoers`, centralizar los registros en un servidor externo y automatizar toda la configuración con un script.

7. Bibliografía

- Shotts, W. (2019). *The Linux Command Line* (2.ª ed.). No Starch Press.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). John Wiley & Sons. Capítulos 16 y 17, "Security" y "Protection".
- Linux Audit Project. (s. f.). *Linux audit documentation*. Recuperado el [dd/mm/aaaa] de https://github.com/linux-audit/audit-documentation
- Debian Project. (s. f.). *Documentación de Debian*. Recuperado el [dd/mm/aaaa] de https://www.debian.org/doc/
- Kerrisk, M. (s. f.). *Linux man-pages project*. Recuperado el [dd/mm/aaaa] de https://man7.org/linux/man-pages/

- Arch Linux Wiki. (s. f.). *Security*. Recuperado el [dd/mm/aaaa] de https://wiki.archlinux.org/title/Security

8. Anexos

- **Anexo A:** capturas de pantalla (Figuras 1 a 9), incluidas en el Caso Práctico.
- **Anexo B:** registro de auditoría completo (Figura 5).
- **Anexo C:** contenido del archivo `/etc/audit/rules.d/acceso.rules`:

```
-w /seguro/datos.txt -p rwxa -k acceso_datos
```